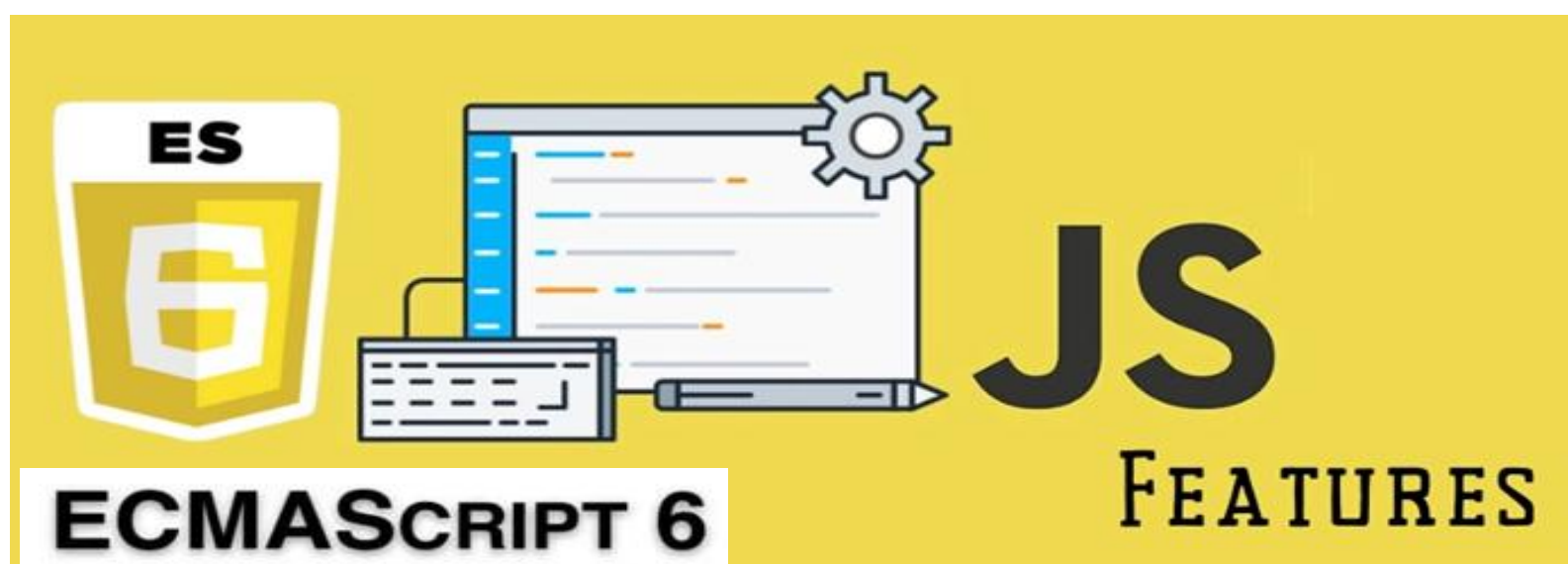




VIT[®]
AP



WEB TECHNOLOGIES

CSE-4004

MODULE 4: ES6: Promise and Async

By: Dr.MD.Sirajuddin

SCOPE

WHAT IS PROMISE?

- A Promise is an object that represents the eventual/future **completion** (or failure) of an **asynchronous operation** and its resulting value.
- A Promise in JavaScript is an object that represents the **result** of an **asynchronous operation** that may complete in the future.
- A promise can be in one of three mutually exclusive states:
 1. **Pending**: The initial state; the asynchronous operation is still running.
 2. **Fulfilled (or Resolved)**: The operation completed successfully, and a value is available.
 3. **Rejected**: The operation failed, and a reason (error) is available.

WHY PROMISE?

- Handle asynchronous operations
- Avoid nested callback functions improves the readability of code
- Better error handling

PROMISE CREATION

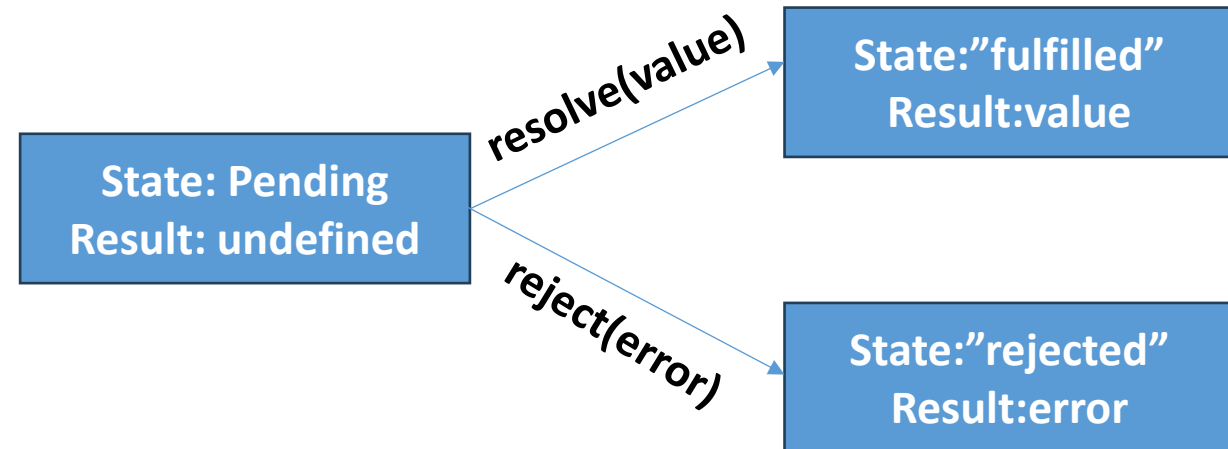
- You create a new promise using the Promise constructor, which takes two functions as parameters: **resolve** and **reject**.
- **.then()** and **.catch()** methods are used to handle the outcome.

```
let promise = new Promise(function(resolve, reject) { async operations });
```

resolve() → success

reject() → failure

```
let myPromise = new Promise(function(resolve, reject) {  
  let success = true;  
  if(success) {  
    resolve("Task completed");  
  } else {  
    reject("Task failed");  
  }  
});
```



Handling Promise using .then()

.then() executes when promise is resolved.

```
myPromise.then(function(result)
```

```
{
```

```
  console.log(result);
```

```
});
```

Output: Task Completed

Handling Errors using .catch()

.catch() executes when promise is rejected.

```
.catch(function(error) {
```

```
  console.log(error);
```

```
});
```

Example:

```
let payment = new
Promise(function(resolve, reject)
{
  let paymentStatus = "success";
  if(paymentStatus === "success")
  {
    resolve("Payment Successful");
  }
  else
  {
    reject("Payment Failed");
  }
});
```

```
payment
.then(function(msg)
{
  console.log(msg);
})
.catch(function(err)
{
  console.log(err);
});
```

PROMISE with TIMER

```
let dataLoad = new Promise(function(resolve, reject)
{
  setTimeout(function()
  {
    resolve("Data loaded from server");
  },2000); //wait for 2000msec or 2sec

});
```

```
dataLoad.then(function(result)
{
  console.log(result);
});
```

After 2 seconds

Data loaded from server

PROMISE CHAINING

Each `.then()` returns a **new Promise**, and the returned value goes to the next `.then()`.

`.then()` → return value → next `.then()`

```
let promise = new Promise(function(resolve) {  
    resolve(10);  
});
```

`promise`

```
.then(function(num) {  
    return num * 2;  
})  
.then(function(result) {  
    console.log(result);  
});
```

Output: 20

Async Keyword

If the **async** keyword is used before a function, then the function will return a **Promise**.

```
async function showMessage()  
{  
  return "Hello Students";  
}
```

```
showMessage().then(function(result)  
{  
  console.log(result);  
});
```

Await Keyword

- await is used inside an async function.
- It pauses the function execution until the Promise is resolved.

Example: let result = await promise;

```
function getData() {  
  return new Promise(function(resolve) {  
    setTimeout(function() {  
      resolve("Data received");  
    }, 2000);  
  });  
}
```

```
async function display(){  
  let result = await getData();  
  console.log(result);  
}
```